PYTHON

Python for Data Analysts

Only the Pandas you need for analyst work, mapped to SQL you already know, with a 30-day plan.

- **Load, clean and analyse in 10 lines**
- **SQL to Pandas cheat table**
- **30-day plan from zero**

@thedataguy16

Load and clean

First 5 minutes with any new dataset. Run these before any analysis.

```
import pandas as pd
df = pd.read_csv("sales.csv")

df.shape              # rows, columns
df.info()             # types and nulls
df.describe()         # quick stats
df.isna().sum()       # nulls per column

df = df.drop_duplicates()
df["date"] = pd.to_datetime(df["date"])
df["amount"] = df["amount"].fillna(0)
```

Check dtypes first. Numbers read as text are the #1 reason a groupby gives wrong totals.

@thedataguy16

Analyse

These 4 lines answer most business questions.

Filter

North region orders above ₹5,000.

```
df.query("region == 'North' and amount > 5000")
```

Group and summarise

Revenue and order count by region.

```
df.groupby("region").agg(
  revenue=("amount","sum"), orders=("id","count"))
```

Join

Add customer details to orders.

```
pd.merge(orders, customers, on="cust_id", how="left")
```

Pivot

Region by month table.

```
df.pivot_table(index="region",
  columns="month", values="amount", aggfunc="sum")
```

@thedataguy16

SQL to Pandas

If you know SQL, you already know 80% of Pandas.

- **WHERE**

```
df[df.amount > 500]   or   df.query()
```

- **GROUP BY**

```
df.groupby("col").agg(...)
```

- **JOIN**

```
pd.merge(a, b, on="id", how="left")
```

- **UNION ALL**

```
pd.concat([a, b])
```

- **ORDER BY ... DESC**

```
df.sort_values("amount", ascending=False)
```

- **ROW_NUMBER**

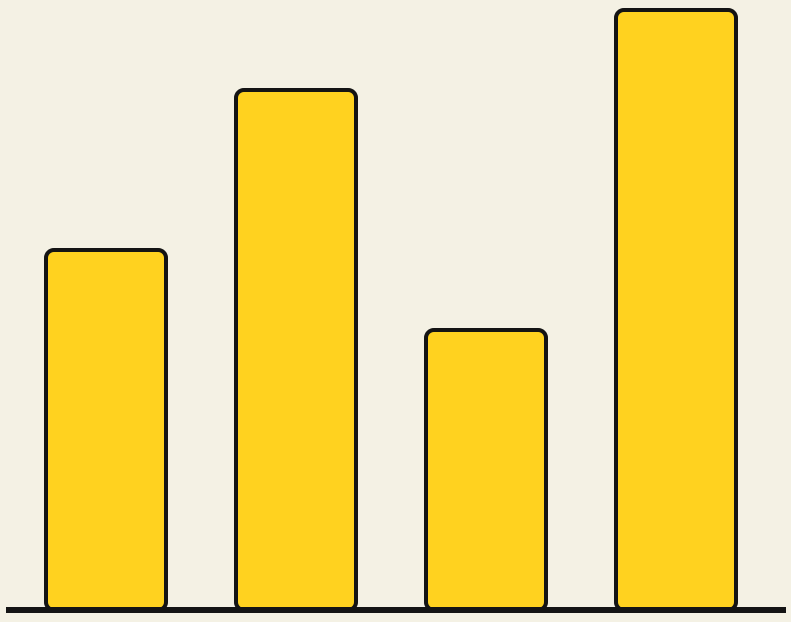
```
df.groupby("g").cumcount() + 1
```

@thedataguy16

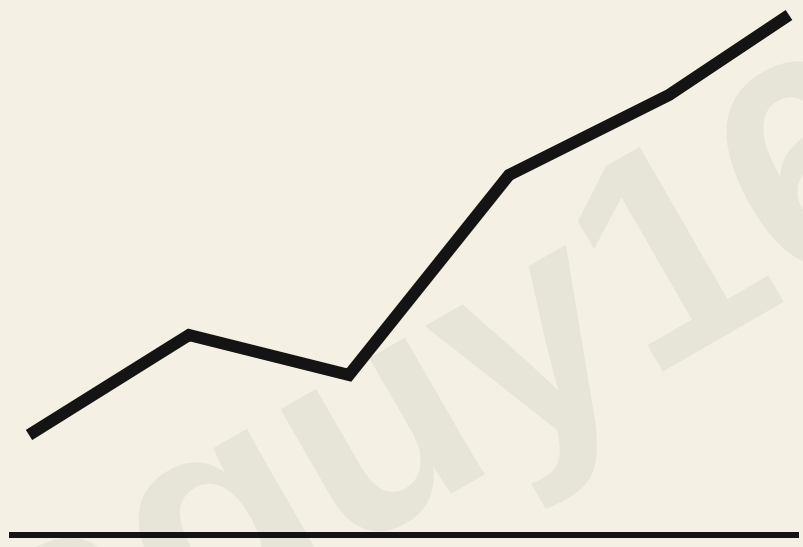
Charts in 3 lines

One chart that answers the question beats ten pretty ones.

plt.bar



plt.plot



```
import matplotlib.pyplot as plt
s = df.groupby("region")["amount"].sum()
s.sort_values().plot(kind="barh")
plt.title("Revenue by region");
plt.show()
```

- **Bar**
  Compare categories.
- **Line**
  Show change over time.
- **Histogram**
  See the spread of one number.

@thedataguy16

Clean messy real data

Real datasets look nothing like course datasets. These fix most issues.

```
# text columns
df["city"] =
df["city"].str.strip().str.title()

# money stored as text "₹1,200"
df["amt"] = (df["amt"].str.replace("
[₹,]", "", regex=True)
                .astype(float))

# remove outliers with IQR
q1, q3 = df["amt"].quantile([.25, .75])
iqr = q3 - q1
df = df[df["amt"].between(q1-1.5*iqr,
q3+1.5*iqr)]

# dates
df["date"] = pd.to_datetime(df["date"],
dayfirst=True)
```

Always print df.shape before and after cleaning. Say how many rows you removed and why.

@thedataguy16

30 days from zero

One hour a day is enough. The project at the end matters more than the course.

- **Days 1-5: Python basics**
  Variables, lists, dictionaries, loops, functions. Skip classes for now.
- **Days 6-17: Pandas**
  Load, clean, filter, groupby, merge, pivot, dates.
- **Days 18-22: Charts**
  Bar, line, histogram with Matplotlib or Seaborn.
- **Days 23-30: One project**
  Real dataset, 5 business questions answered, pushed to GitHub with a clear README.

@thedataguy16

10 interview questions

With the one line answer interviewers want to hear.

- **loc vs iloc**
  loc uses labels, iloc uses positions.
- **merge vs concat**
  merge joins on keys, concat stacks tables.
- **apply vs vectorised**
  Vectorised is much faster. Use apply only when needed.
- **Missing values?**
  Check why they are missing, then drop, fill or flag.
- **Huge CSV?**
  Read in chunks, pick columns with usecols, set dtypes.
- **List vs tuple**
  List can change, tuple cannot.

@thedataguy16